



[Услуги](#) [Обо мне](#) [Кейсы](#) [База Знаний](#) [Статьи](#) [FAQ](#)

[Контакты](#)

Косинусная близость

[Продвижение сайтов](#) [База Знаний](#) [Косинусная близость](#)

З А В С D E F G H I J K L M N O P Q R S T U V W

Х Y А В Г Д З И **К** Л М Н О П Р С Т Ф Х Ч Ш Э Ю

Ка Кл **Ко** Ку Кэ

Кол Кон Кор **Кос** Коэ

Обновлено: 19.06.2025

Косинусная близость — это мера сходства между двумя векторами, которая вычисляется на основе косинуса угла между ними, или, что эквивалентно, скалярное произведение между их нормализациями.. Она широко используется в различных областях, таких как обработка естественного языка, информационный поиск и рекомендательные системы, для определения степени похожести текстов, документов или других объектов, представленных в векторном виде.

Содержание [[скрыть](#)]

- 1 Как это работает
- 2 Формула для расчета косинусной близости
- 3 Пример использования для сравнения текстов
- 4 Как косинусная близость используется поисковыми системами
 - 4.1 Пример
- 5 Преимущества использования косинусной близости в поисковых системах
- 6 Текстово-семантический анализ с косинусной близостью на Python
 - 6.1 Интерпретация результатов
 - 6.2 Дополнительные шаги для SEO-специалиста
 - 6.3 Важно помнить
 - 6.4 Связанные термины

Как это работает

Фактический анализ доступных исходников кода свидетельствует о том, что Google использует скалярное произведение нормализованных векторов для вычисления сходства.

Если векторы нормализуются до единичной длины (величина = 1), скалярное произведение математически эквивалентно косинусному подобию.

Косинусное подобие = $(A \cdot B) / (|A| \cdot |B|)$

Когда $|A| = |B| = 1$ (нормализованные векторы), это упрощается до:

Косинусное подобие = $A \cdot B$ = скалярное произведение

Таким образом, код эффективно реализует косинусное подобие следующим образом:

- Обеспечение нормализации всех векторов до единичной длины
- Вычисление скалярного произведения между этими нормализованными векторами

Этот подход вычислительно более эффективен, чем вычисление полной формулы косинусного подобию, потому что исключает операцию деления с тем же результатом.

Представьте два вектора, исходящих из начала координат. Косинус угла между ними отражает, насколько эти векторы направлены в одну сторону. Значение косинусной близости находится в диапазоне от -1 до 1:

- **1** означает полное совпадение векторов (угол между ними равен 0 градусов).
- **0** означает, что векторы ортогональны, то есть никак не связаны между собой (угол между ними равен 90 градусов).
- **-1** означает, что векторы направлены в противоположные стороны (угол между ними равен 180 градусов).

Формула для расчета косинусной близости

$$\cos(\theta) = (A \cdot B) / (||A|| * ||B||)$$

где:

- **A** и **B** – векторы, близость которых нужно вычислить.
- **•** – скалярное произведение векторов.
- **||A||** и **||B||** – длины (модули) векторов A и B соответственно.

Пример использования для сравнения текстов

Представим два предложения, которые мы хотим сравнить:

- Предложение 1: “Я люблю читать книги.”
- Предложение 2: “Чтение книг доставляет мне удовольствие.”

Каждое предложение можно преобразовать в вектор, используя, например, метод “**Мешок слов (Bag of Words)**”, где каждый элемент вектора соответствует слову, а его значение отражает частоту встречаемости этого слова в предложении. Затем, вычисляя косинус угла между векторами, можно определить степень смысловой близости предложений.

Косинусная близость – это мощный инструмент для определения сходства между объектами, представленными в векторном пространстве. Благодаря своей простоте и эффективности, она нашла широкое применение в различных областях, связанных с анализом данных и машинным обучением.

Как косинусная близость используется поисковыми системами

Косинусная близость играет важную роль в современных поисковых системах, помогая предоставлять релевантные результаты поиска по текстовым запросам. Вот как это работает:

1. Преобразование текста в векторы:

- И **поисковый запрос** пользователя, и документы в индексе поисковой системы преобразуются в числовые векторы.
- Каждый элемент вектора представляет собой вес определенного слова или термина в тексте.
- Существуют разные методы векторизации, например, **TF-IDF** (Term Frequency-Inverse Document Frequency), где вес слова тем выше, чем чаще оно встречается в документе, но реже в целом **корпусе** документов.

2. Расчет косинусной близости:

- После того как запрос пользователя и документы представлены в виде векторов, вычисляется косинусная близость между вектором запроса и вектором каждого документа.
- Чем ближе косинусная близость к 1, тем больше сходство между запросом и документом, что указывает на более высокую **релевантность** документа к запросу.

3. Ранжирование результатов:

- **Поисковая система** сортирует документы в порядке убывания косинусной близости к запросу пользователя.
- Документы с наибольшей косинусной близостью, то есть наиболее релевантные, отображаются вверху страницы результатов поиска.

Пример

Представьте, что пользователь вводит запрос “рецепты пиццы”. Поисковая система преобразует этот запрос в вектор. Затем она сравнивает этот вектор с векторами документов в своем индексе, например:

- **Документ 1:** “Лучшие рецепты итальянской пиццы” – вероятно, будет иметь высокую косинусную близость к запросу.
- **Документ 2:** “История пиццы: от Древнего Рима до наших дней” – может иметь более низкую косинусную близость, так как фокусируется на истории, а не на рецептах.

- **Документ 3:** “Как приготовить тесто для пиццы” – также может иметь высокую косинусную близость, хотя и не содержит слова “рецепт”.

Преимущества использования косинусной близости в поисковых системах

- Учет семантической близости. Даже если документы не содержат точных слов из запроса, косинусная близость позволяет определить их релевантность на основе общих терминов и [контекста](#).
- Независимость от длины документа. Косинусная близость нормализует длину векторов, что позволяет сравнивать документы разной длины без искажений – например, можно сопоставить поисковый запрос и весь текст посадочной страницы на сайте.
- Косинусная близость используется для оценки релевантности не только текстовых данных, но и соответствия контента профилю пользователя (персонализация результатов поиска), оценки тематичности, спамности и т.п.
- Эффективность. Существуют алгоритмы, позволяющие быстро вычислять косинусную близость для больших наборов данных.

Таким образом, косинусная близость является важным инструментом в арсенале современных поисковых систем, помогая им понимать смысл запросов пользователей и находить наиболее релевантную информацию.

Текстово-семантический анализ с косинусной близостью на Python

Этот пример демонстрирует, как [SEO](#)-специалист может использовать косинусную близость для анализа текстовой семантики двух страниц сайта.

Шаг 1. Импорт необходимых библиотек:

```
import nltk
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download('punkt')
nltk.download('stopwords')
```

Шаг 2. Получение текстового контента страниц:

```
# Здесь нужно заменить "url1" и "url2" на реальные адреса страниц
сайта.
# Функция get_text_from_url не приведена, так как способ получения
текста с сайта
# может варьироваться (парсинг, API и т.д.).

text1 = get_text_from_url("url1")
text2 = get_text_from_url("url2")
```

Шаг 3. Предобработка текста:

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('russian'))

def preprocess_text(text):
    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word.isalnum() and word not
in stop_words]
    return " ".join(tokens)

processed_text1 = preprocess_text(text1)
processed_text2 = preprocess_text(text2)
```

Шаг 4. Векторизация текста с помощью TF-IDF:

```
texts = [processed_text1, processed_text2]
```

```
vectorizer = TfidfVectorizer()  
tfidf_matrix = vectorizer.fit_transform(texts)
```

Шаг 5. Вычисление косинусной близости:

```
cosine_sim = cosine_similarity(tfidf_matrix[0], tfidf_matrix[1])  
print(f"Косинусная близость: {cosine_sim[0][0]}")
```

Интерпретация результатов

- Значение косинусной близости будет находиться в диапазоне от 0 до 1.
- **Ближе к 1:** Высокая семантическая близость текстов на страницах. Это может свидетельствовать о дублировании контента или о том, что страницы охватывают очень схожие темы.
- **Ближе к 0:** Низкая семантическая близость, что указывает на то, что страницы, вероятно, относятся к разным темам.

Косинусное подобие учитывает угол, но не длину векторов. И если мы используем вложения, где величина является полезным элементом представления текста, **результаты оценки будут некорректны.**

Дополнительные шаги для SEO-специалиста

- Визуализация матрицы косинусной близости для анализа большого количества страниц.
- **Кластеризация** страниц на основе косинусной близости для определения групп схожих тем.
- Анализ ключевых слов с наибольшим вкладом в косинусную близость.

Важно помнить

- Косинусная близость – это всего лишь один из инструментов для анализа текстов.
- Важно интерпретировать результаты в контексте конкретного сайта и его целей.
- Для более глубокого анализа семантики рекомендуется использовать более продвинутые методы **NLP**, такие как векторные вложения (**эмбединги**) или модели

трансформеров.

Связанные термины

- Тематическое моделирование
- BERTopic
- NLP
- SBERT
- Site2Vec

LLM NLP python алгоритмы графы машинное обучение методы семантика текстовые метрики



Искать

Q

Новые статьи

- GEO, AEO, AISEO и почему не работают традиционные методы SEO
- Перенос сайта на новый домен: комплексное руководство
- Семантический поиск 2025: от ключевых слов к смыслу
- Как писать экспертные статьи, если вы не эксперт
- Семантика: пересматриваем теорию и практику





info@textarget.ru

О сайте

Поисковая оптимизация для коммерческих и информационных проектов.



Услуги

Поисковый маркетинг

SEO

Контент-маркетинг

SEO-Аудит

Статьи

SEO-блог